

Proyecto 1 Introducción a los Repositorios de Código Distribuido:

UOH Survivors

Felipe Scanda Martínez
Ingeniería Civil en Computación
Universidad de O'Higgins
05 de julio de 2026

<https://github.com/FelipeScanda/UOH-Survivors/>

Índice

<i>Descripción del proyecto</i>	<i>3</i>
<i>Estructura del repositorio</i>	<i>4</i>
<i>Evolución del desarrollo</i>	<i>4</i>
<i>Análisis del historial de Git.....</i>	<i>7</i>
<i>Análisis de integración de cambios</i>	<i>8</i>
<i>Análisis de métricas</i>	<i>10</i>
<i>Reflexión final</i>	<i>11</i>

Descripción del proyecto

Vampire Survivors es un videojuego del género “roguelike” y “survival action” desarrollado por Luca Galante, lanzado originalmente en 2021. El juego se caracteriza por ofrecer partidas rápidas y dinámicas en las que el jugador debe sobrevivir a oleadas constantes de enemigos mientras mejora progresivamente las habilidades de su personaje. A pesar de presentar una estética visual sencilla inspirada en videojuegos retro de 8 y 16 bits, el título destaca por su jugabilidad adictiva y su sistema de progresión estratégico.

La mecánica principal consiste en controlar a un personaje dentro de un escenario abierto donde los enemigos aparecen de manera continua y aumentan su dificultad con el paso del tiempo. El jugador puede desplazarse libremente por el mapa, mientras que los ataques se ejecutan automáticamente, permitiendo enfocar la experiencia en la movilidad, la evasión y la toma de decisiones respecto a las mejoras disponibles. Durante cada partida, es posible obtener nuevas armas, habilidades pasivas y mejoras que pueden combinarse para generar efectos más poderosos.

Uno de los aspectos más relevantes de Vampire Survivors es su sistema de progresión incremental. A medida que el jugador avanza y derrota enemigos, obtiene experiencia y recursos que permiten desbloquear personajes, escenarios y mejoras permanentes para futuras partidas. Esto genera un ciclo de juego basado en el aprendizaje y la optimización de estrategias, incentivando la experimentación con diferentes combinaciones de habilidades.

Para este proyecto, se realizará una versión minimalista de este videojuego, utilizando la temática de UOH, principalmente donde el personaje principal corresponde a la mítica mascota de la universidad “Barranquin”.

Se tiene como objetivo implementar las mecánicas principales del videojuego original, como lo pueden ser disparar automáticamente, mover al personaje principal por el mapa y mejorando sus estadísticas, un sistema de enemigos por oleadas donde la dificultad sea incremental, es decir, que los enemigos se vayan volviendo más rápidos y con más vida a través del tiempo, con distintos tipos de enemigos. Además, se tiene en consideración una interfaz gráfica, como el menú principal que da acceso a jugar o modificar el aspecto visual del jugador, y la interfaz dentro del juego, la cual da información acerca de la vida del jugador, experiencia y tiempo sobrevivido.

Estructura del repositorio

El repositorio se encuentra compuesto con los siguientes archivos y carpetas:

- assets: Incluye los archivos gráficos del juego, como, por ejemplo, imágenes con el aspecto visual del jugador, enemigos, mapa, etc.
- entities: Contiene archivos relacionados con las entidades que conforman el juego, como el jugador, enemigos, gemas de experiencia, proyectiles, etc. En estos archivos se encuentran las clases de dichas entidades, las cuales se importan dentro del archivo principal.
- items: Contiene archivos relacionados con los ítems que se pueden obtener dentro del juego. Al igual que el anterior, cada archivo contiene una clase que se importa en el juego principal.
- .gitignore: Aquí se encuentran los nombres de archivos y carpetas que no se quieren considerar dentro del repositorio, como archivos de cache de Python y el entorno virtual.
- main.py: Corresponde al archivo principal del juego. Este archivo ejecuta el juego como tal, e importa todas las clases necesarias. También incluye toda la lógica del juego e interfaces gráficas.
- README.md: En este archivo se especifica la idea principal del juego, los requerimientos, el paso a paso para la instalación y cómo ejecutar el juego, etc. También incluye una lista de futuras implementaciones propuestas.
- settings.py: Este archivo contiene toda la configuración del juego, como el tamaño de la ventana, FPS, etc.

Evolución del desarrollo

Al empezar el desarrollo del juego, se inició implementando la ventana principal del juego, para luego ir implementando las demás funciones dentro de esta ventana. Luego de la ventana, se implementó el jugador y su movimiento dentro de la ventana, haciendo los ajustes necesarios para tener un movimiento uniforme en todas las direcciones.

Luego de implementado el jugador, se necesitaban enemigos los cuales persiguieran al jugador, calculando la dirección en la cual estos se debían mover para llegar a la posición del jugador.

Después, el jugador necesitaba una herramienta con la cual poder derrotar a los enemigos, con lo que se implementaron los proyectiles, los cuales debían salir de la posición del jugador hasta la posición donde está el enemigo más cercano.

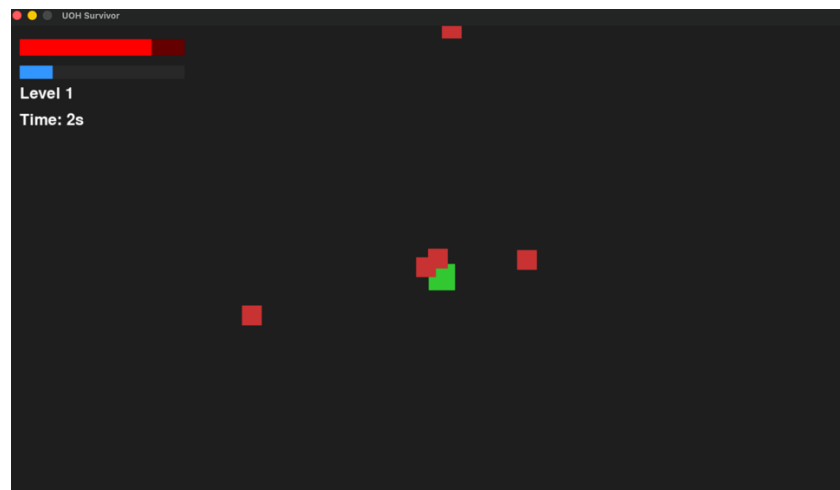
También se debía detectar cuándo estos proyectiles impactaban con los enemigos, por lo que primero se implementó un nivel de vida en los enemigos. Luego, los enemigos debían soltar alguna recompensa para el jugador, por lo que se programó la experiencia y el nivel de experiencia en el jugador, así una vez el enemigo recibiera daño y muriese, soltaría una gema de experiencia. Finalmente, se implementó la colisión de los proyectiles en los enemigos y el dropeo de experiencia.

Sin embargo, al inicio solo había 5 enemigos dentro del juego, por lo que se añadió el spawn constante de enemigos. Así, se implementó que cuando el jugador mataba los suficientes enemigos, podía subir de nivel y elegir una mejora, pero seguía quedando otro problema, el jugador era inmortal y no podía recibir daño, así que también implementó la función de recibir daño y las colisiones de enemigos con el jugador, de manera que cuando la vida del jugador llegara a 0, se terminara el juego.

Con esto, el juego era bastante jugable, pero hacía falta algo que le diera información al jugador del estado de la partida, por lo que se agrega la interfaz gráfica de la cantidad de vida, experiencia, y también el menú de subir de nivel, dándole 3 opciones de mejora al jugador.

Después de esto, se tenía un juego completamente jugable, pero era demasiado fácil, ya que los enemigos siempre tenían la misma vida y velocidad, así que se implementó el escalado de enemigos para que el juego fuese más difícil a través del tiempo, y también en la interfaz gráfica se agregó el tiempo de juego.

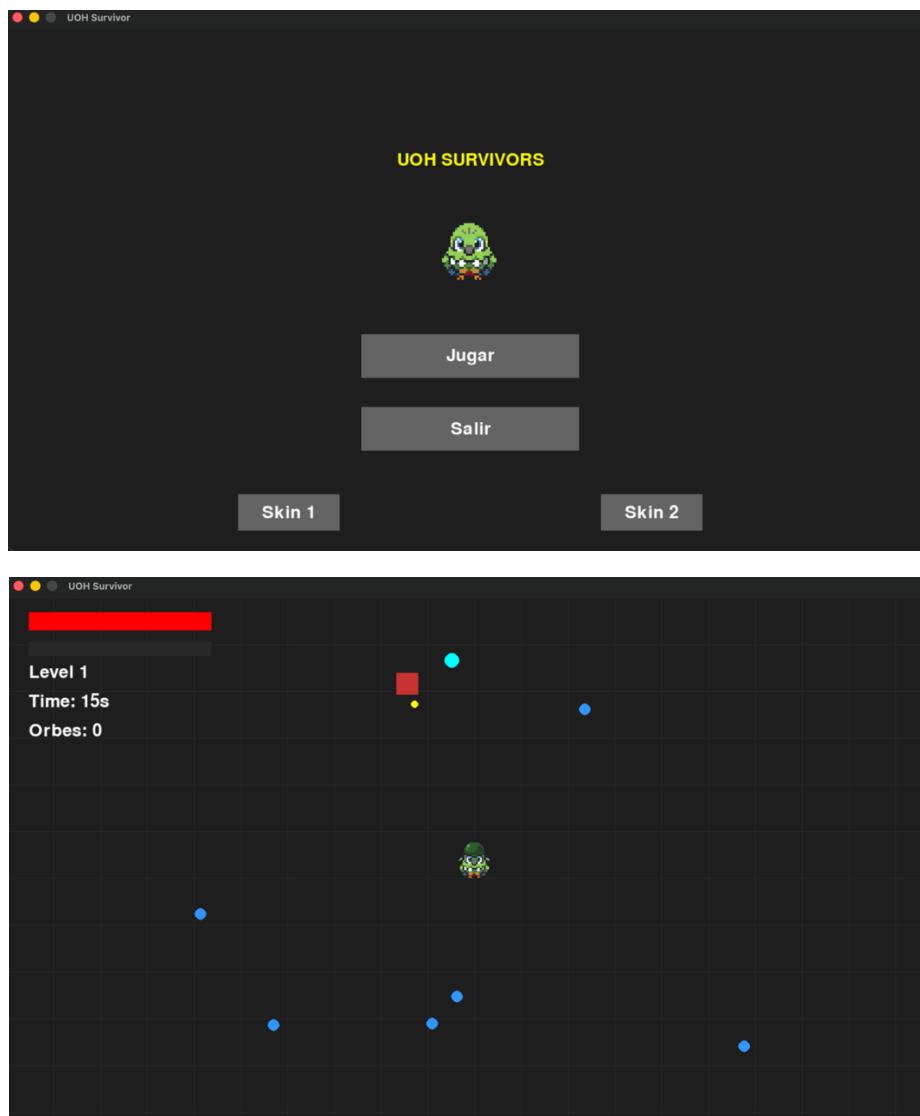
Con todo esto, se llega a la versión jugable 1.0 del juego, que se veía más o menos así:



Hasta este punto, el juego era jugable, pero se sentía muy uniforme, así que se implementaron otros 2 tipos de enemigos, con distintas características, para brindarle más variedad al juego, y también un nuevo tipo de arma, el orbe, el cual es un ítem

dropeable por los enemigos, y que al recogerlo, se pondría a orbitar alrededor del jugador y hacer daño a los enemigos cercanos, y también una cámara infinita que sigue al jugador para otorgarle más libertad de movimiento, con lo que después de corregir algunos bugs y parámetros, se agregó una grilla al mapa de fondo para dar más sensación de movimiento.

Finalmente, se ajustaron algunos aspectos gráficos, como añadir el sprite del jugador, agregar la interfaz del menú principal, menú de pausa y gameover, selección de skins, entre otras modificaciones que pulieron el juego un poco más, quedando de la siguiente manera:



Con esto se completó el desarrollo de momento, ya que aún se espera seguir implementando funciones, mejorar gráficos, música, entre otras mejoras que se encuentran dentro del archivo README.md

Análisis del historial de Git

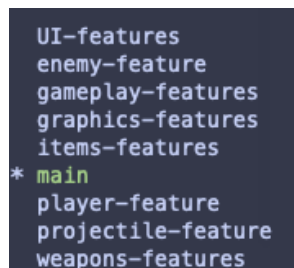
El historial de Git contiene todos los commits realizados a lo largo del proyecto, los cuales hasta el momento de este informe son 55, los cuales se muestran a continuación:

Graph	Description	Date	Author	Commit
	main origin origin/HEAD Agrega instrucciones de juego al README.md	5 Jul 2026 19:08	Felipe Scanda	81f666b9
	Actualiza README.md	5 Jul 2026 18:05	Felipe Scanda	5aaa6085
	Resuelve conflicto entre UI-features y graphics-features	5 Jul 2026 17:43	Felipe Scanda	d5b415fb
	Mergea rama 'UI-features'	5 Jul 2026 17:37	Felipe Scanda	43d6c25b
	graphics-features origin Modifica el color de la barra de vida y formatea su codigo de main.py	5 Jul 2026 17:36	Felipe Scanda	ae053c59
	UI-features origin Mueve la posicion de la UI y formatea su codigo de main.py	5 Jul 2026 17:31	Felipe Scanda	11f80197
	Ajusta el tamaño del sprite del jugador	5 Jul 2026 17:13	Felipe Scanda	6a89fbe3
	Implementa seleccion de skins en menu principal	5 Jul 2026 16:53	Felipe Scanda	9a3879d3
	Agrega variable para seleccionar skin	5 Jul 2026 16:37	Felipe Scanda	6478cbe5
	Mergea rama 'main' en graphics-features	5 Jul 2026 15:08	Felipe Scanda	6b8a4bbb
	Agrega los sprites del jugador	5 Jul 2026 15:08	Felipe Scanda	61906ef3
	Elimina el renderizado del gameplay en el menu principal	4 Jul 2026 18:13	Felipe Scanda	391dae2b
	Añade interfaz de menu principal, menu de pausa y pantalla de gameover	4 Jul 2026 18:06	Felipe Scanda	2ed223f3
	Agrega grid al mapa	4 Jul 2026 17:38	Felipe Scanda	779d2267
	weapons-features origin Aumenta la distancia de giro de los orbes	4 Jul 2026 17:30	Felipe Scanda	a2790212
	gameplay-features origin Elimina los 5 enemigos default	4 Jul 2026 17:26	Felipe Scanda	0a579774
	Soluciona bugs del temporizador y spawn de enemigos	4 Jul 2026 17:24	Felipe Scanda	c8a003aa
	Implementa camara infinita que sigue al jugador	4 Jul 2026 17:10	Felipe Scanda	a90b1a1e
	Implementa redistribucion de orbes para que aparezcan simetricamente alrededor del jugador	4 Jul 2026 14:03	Felipe Scanda	34f4e7a0
	Modifica la distancia de giro de los orbes	4 Jul 2026 13:57	Felipe Scanda	9a00bc81
	Agrega linea eliminada en el commit anterior	4 Jul 2026 13:52	Felipe Scanda	1c8f0b74
	Implementa los items de orbe para ir agregando orbes a medida que se recogen	4 Jul 2026 13:47	Felipe Scanda	d71cc715
	items-features origin Agrega el item de orbe	4 Jul 2026 13:35	Felipe Scanda	855fdee2
	Agrega el arma de orbe giratorio	4 Jul 2026 13:27	Felipe Scanda	ef001ef0
	enemy-feature origin Agrega distintos tipos de enemigos	29 Jun 2026 17:01	Felipe Scanda	cadafe12
	Formatea codigo de main.py (Version jugable 1.0 del juego)	29 Jun 2026 16:09	Felipe Scanda	602664b8
	Agrega tiempo de partida a la UI	29 Jun 2026 15:56	Felipe Scanda	fe7281e4
	Agrega escalado a los enemigos	29 Jun 2026 15:53	Felipe Scanda	425acafc
	Mergea rama de 'UI-features'	29 Jun 2026 15:32	Felipe Scanda	41bbebd8
	Actualiza la interfaz grafica	28 Jun 2026 20:02	Felipe Scanda	0c729dc9
	Agrega funcionalidad de menu de level up	28 Jun 2026 19:52	Felipe Scanda	29170104
	Agrega interfaz con vida, nivel y XP	27 Jun 2026 19:03	Felipe Scanda	0522412f
	Mergea rama de 'player-feature'	27 Jun 2026 18:59	Felipe Scanda	87829d4a
	player-feature origin Agrega condicion de gameover	27 Jun 2026 18:51	Felipe Scanda	6d3bf8a6
	Agrega colisiones de enemigos con el jugador	27 Jun 2026 18:51	Felipe Scanda	7978ab9f
	Agrega propiedades de vida y funcion de recibir daño al jugador	27 Jun 2026 18:45	Felipe Scanda	88c696b6
	Añade sistema de recolección de experiencia a main.py	26 Jun 2026 18:54	Felipe Scanda	3e50f1e2
	Añade metodo para subir de nivel a player.py	26 Jun 2026 18:47	Felipe Scanda	046e436b
	Añade spawn constante de enemigos a main.py	26 Jun 2026 18:41	Felipe Scanda	41e266d2
	Implementa la colision de proyectiles y caída de experiencia en main.py	26 Jun 2026 18:33	Felipe Scanda	61b95bf3
	Mergea rama 'enemy-feature' en gameplay-features	26 Jun 2026 18:24	Felipe Scanda	5c3e5629
	Agrega a enemigo funcion de recibir daño	26 Jun 2026 18:19	Felipe Scanda	3e1bc0f0
	Agrega entidad de experiencia y añade las propiedades al jugador	26 Jun 2026 18:17	Felipe Scanda	fd5bfaef
	Mergea la rama 'projectile-feature' sin fast-forward	26 Jun 2026 11:52	Felipe Scanda	a9aefb02
	projectile-feature origin Agrega vida a la entidad enemigo	26 Jun 2026 11:51	Felipe Scanda	ed1cdaf3
	Agrega la entidad de proyectil y la implementa en el main y en la entidad player	26 Jun 2026 11:51	Felipe Scanda	65531262
	Agrega la entidad de enemy y la implementa en el main	25 Jun 2026 22:45	Felipe Scanda	57622ad3
	Modifica player.py para normalizar el movimiento del jugador	25 Jun 2026 22:28	Felipe Scanda	66f116ed
	Agrega la entidad de player y la implementa en el main	25 Jun 2026 22:16	Felipe Scanda	5c7af86d
	Agrega la estructura principal del juego a main.py (ventana)	25 Jun 2026 19:30	Felipe Scanda	d8aef2b6
	Actualiza .gitignore para ignorar el cache de python	25 Jun 2026 19:29	Felipe Scanda	04016425
	Agrega el archivo settings.py con las configuraciones del juego	25 Jun 2026 19:25	Felipe Scanda	7d23f461
	Actualiza el README.md con los requerimientos	25 Jun 2026 19:20	Felipe Scanda	4fab5b2
	Agrega .gitignore	25 Jun 2026 19:16	Felipe Scanda	a45d55b8
	Primer commit	25 Jun 2026 19:02	Felipe Scanda	2802c802

Los días con actividad dentro del repositorio fueron el viernes 26, sábado 27, domingo 28, lunes 29 de junio, y el sábado 4 y domingo 5 de julio, por motivos de tiempo, no se logró un desarrollo constante, pero sí se lograron los 6 días de desarrollo solicitado.

Uno de los archivos más modificados fue el main.py, ya que es el archivo principal del juego y en él se encuentran la mayoría de las funcionalidades relacionadas a éste, sin embargo, otro que también fue bastante modificado fue el de player.py, ya que se requerían ir agregando funcionalidades relacionadas a las mecánicas y comportamiento del jugador dentro del juego.

Para el desarrollo del proyecto, se utilizaron 9 ramas, cada una destinada a implementar una funcionalidad dentro del juego, así, cada cambio realizado se hacía en una rama, y al finalizar y comprobar que el juego funcionaba de manera correcta, se llevaba dicho cambio a la rama main con un merge. Las ramas utilizadas son las siguientes:



```
UI-features
enemy-feature
gameplay-features
graphics-features
items-features
*main
player-feature
projectile-feature
weapons-features
```

La mayoría de los merge fueron sin fast-forward, lo que implicó que el historial de commits se mantuviera en su mayoría con una forma lineal, sin embargo, en algunos puntos, no fue así, como, por ejemplo, en el commit “a9aefb02”, el cual no utilizó fast-forward, sino que fue un three-way merge.

Análisis de integración de cambios

Durante el desarrollo, como se mencionó anteriormente, se utilizaron distintas ramas, cada una con su propio objetivo dentro del juego:

- UI-features: Destinada a realizar implementaciones de interfaz gráfica.
- enemy-feature: Destinada a realizar la implementación y cambios de los enemigos.
- gameplay-features: Destinada a realizar implementaciones de aspectos del gameplay, como, por ejemplo, subida de nivel, recolección de experiencia, escalado de enemigos, entre otros.

- graphics-features: Destinada a implementar aspectos gráficos del juego, como sprites.
- items-features: Destinada a implementar los ítems dropeables dentro del juego y su comportamiento.
- main: Rama principal de desarrollo del juego.
- player-feature: Destinada a la implementación y cambios del jugador.
- projectile-feature: Destinada a la implementación y cambios de los proyectiles.
- weapons-features: Destinada a implementar las distintas armas que el jugador puede utilizar dentro del juego.

Dentro de las distintas ramas, hubo un punto que al hacer un merge de 2 de ellas a la rama main, las cuales son UI-features y graphics-features, se llegó a un conflicto, ya que estas 2 ramas cumplen una función similar. Particularmente, en ambas se modificó la misma sección de código dentro de main.py, que estaba destinada a la interfaz gráfica dentro del gameplay, cambiando la posición de esta en UI-features, y el color de la barra de vida en graphics-features, obteniendo los siguientes conflictos:

```
<<<<<<< HEAD
    pygame.draw.rect(screen, (40, 40, 40), (30, 60, bar_width, 20))

    pygame.draw.rect(screen, (50, 150, 255), (30, 60, bar_width * xp_ratio, 20))
=====
    pygame.draw.rect(screen, (40, 40, 40), (20, 60, bar_width, 20))

    pygame.draw.rect(screen, (50, 150, 255), (20, 60, bar_width * xp_ratio, 20))
>>>>>>> graphics-features
```

```
<<<<<<< HEAD
    pygame.draw.rect(screen, (100, 0, 0), (30, 20, bar_width, bar_height))

    #Vida actual
    pygame.draw.rect(screen, (255, 0, 0), (30, 20, bar_width * health_ratio, bar_height))
=====
    pygame.draw.rect(screen, (100, 0, 0), (20, 20, bar_width, bar_height))

    #Vida actual
    pygame.draw.rect(screen, (200, 40, 40), (20, 20, bar_width * health_ratio, bar_height))
>>>>>>> graphics-features
```

Luego de analizar el conflicto, se optó por conservar los cambios de la rama UI-features, que en ese momento había sido mergeada en la rama main (y luego de tratar

de mergear graphics-features dentro de main, se llegó al conflicto), por lo que se borraron los cambios relacionados a graphics-features:

```
<<<<<< HEAD
pygame.draw.rect(screen, (100, 0, 0), (30, 20, bar_width, bar_height))

#Vida actual
pygame.draw.rect(screen, (255, 0, 0), (30, 20, bar_width * health_ratio, bar_height))
=====

#Barra de XP
```

```
<<<<<< HEAD
pygame.draw.rect(screen, (40, 40, 40), (30, 60, bar_width, 20))

pygame.draw.rect(screen, (50, 150, 255), (30, 60, bar_width * xp_ratio, 20))
=====

#Nivel
```

Finalmente, se borraron los símbolos residuales del conflicto, se agregó el archivo al staging área, y se realizó el commit, quedando como resultado el commit “d5b415fb” y resolviendo el conflicto.

Análisis de métricas

Contando la frecuencia de cambios de cada archivo dentro del repositorio, se obtuvo lo siguiente:

```
32 main.py
10 entities/player.py
6 entities/orbiting_orb.py
6 entities/enemy.py
4 README.md
2 items/orb_item.py
2 entities/xp_gem.py
2 entities/projectile.py
2 .gitignore
1 settings.py
1 assets/player/Barranquin2.png
1 assets/player/Barranquin1.png
```

Con esto podemos ver que el archivo más modificado fue main.py, lo que indica que éste corresponde a un hotspot, lo cual hace sentido, ya que es el archivo principal del juego, donde se incluye toda la lógica y las interfaces. Esto podría ser algo negativo, ya que, al estar incluyendo toda la lógica del juego, interfaces, importación de clases, y demás a un mismo archivo, no se tiene una separación de responsabilidades correcta en el proyecto, lo cual lo hace propenso a problemas. Una solución sería separar las distintas partes de main.py en otros archivos, de manera que cada uno de ellos tenga

una responsabilidad única, dando acceso a un mejor mantenimiento del código, solución de errores más fácil, entre otros.

Reflexión final

Luego de todo el proyecto, además de haber aprendido sobre la programación de videojuegos, se aprendió bastante sobre la herramienta de Git, la cual permitió una correcta gestión de versiones, mejorando el flujo de trabajo y evitando potenciales errores dentro del desarrollo. También, al tener varias ramas destinadas al desarrollo de cada una de las funcionalidades, permitió gestionarlas e implementarlas de manera correcta, asegurando que el juego siguiera funcionando correctamente luego de la implementación, y en caso de que no, permitiendo volver a un punto anterior donde funcionaba correctamente, ahorrando tiempo de desarrollo.

A modo de conclusión, se puede decir que Git es una herramienta fundamental para cualquier desarrollador, ya sea de videojuegos, analista de datos, desarrollador de software, etc. El uso de Git permite gestionar versiones de forma eficiente, tener un historial de respaldo en caso de errores dentro del código o querer revisar una funcionalidad implementada anteriormente y que luego se cambió. Personalmente, este trabajo me permitió aplicar Git de manera práctica en un proyecto, de manera que pude mejorar mis habilidades como desarrollador, adquiriendo una habilidad que es realmente valiosa dentro de la industria.